

به نام خدا

آزمایشگاه سیستم‌عامل - پاییز 1405

پروژه اول: آشنایی با هسته‌ی سیستم‌عامل xv6

طراحان: کسری کاشانی، البرز محمودیان



مقدمه

سیستم‌عامل xv6 یک سیستم‌عامل آموزشی است که در سال 2006 توسط محققان دانشگاه MIT طرح و توسعه داده شده است. این سیستم‌عامل عمدتا به زبان C نوشته شده و بازپیاده‌سازی ساده‌ای از UNIX Version 6 برای معماری x86 است. سیستم‌عامل xv6 علی‌رغم سادگی و حجم کم، دربردارنده نکات حائز اهمیتی در طراحی بوده و برای مقاصد آموزشی بسیار پرکاربرد است. در این پروژه، ضمن آشنایی با معماری و برخی نکات پیاده‌سازی این سیستم‌عامل، آن را اجرا و اشکال‌زدایی خواهید کرد و همچنین برنامه‌ای در سطح کاربر نوشته خواهد شد که روی آن قابل اجرا باشد.

آشنایی با سیستم‌عامل و xv6

کدهای مربوط به سیستم‌عامل xv6 از لینک زیر قابل دسترسی است:
https://github.com/mit-pdos/xv6-public
همچنین مستندات این سیستم‌عامل در صفحه درس بارگذاری شده است. برای این پروژه، نیاز است که فصل‌های صفر و یک و پیوست ب از مستندات فوق را مطالعه کرده و به سوالات داده‌شده پاسخ دهید. پاسخ این سوالات را در قالب یک گزارش بارگذاری خواهید کرد. همچنین توصیه می‌شود برای آشنایی بیشتر، پیوست الف را نیز مطالعه کنید.
سیستم‌عامل جزو نخستین نرم‌افزارهایی است که پس از روشن شدن سیستم اجرا می‌گردد. این نرم‌افزار، رابط نرم‌افزارهای کاربردی با سخت‌افزار رایانه است.

1) سه وظیفه اصلی سیستم عامل را نام ببرید.
2) آیا وجود سیستم‌عامل در تمام سیستم‌های کامپیوتری الزامی است؟ چرا؟ سیستم‌عامل چه مزایایی نسبت به دسترسی مستقیم برنامه‌ها به منابع سخت‌افزاری فراهم می‌کند؟
3) چه معماری‌هایی برای سازمان‌دهی هسته[1]ی سیستم‌عامل وجود دارد؟ هر یک را به اختصار توضیح دهید. سپس مشخص کنید xv6 از نظر سازمان‌دهی هسته دارای چه معماری‌ای است.

[1] Kernel organization

4) سیستم‌عامل xv6 یک سیستم تک‌وظیفه‌ای[2] است یا چندوظیفه‌ای[3]؟
5) چه تفاوتی میان یک برنامه[4] و یک پردازه[5] وجود دارد؟
6) ساختار یک پردازه در سیستم‌عامل xv6 از چه بخش‌هایی تشکیل شده است؟ این سیستم‌عامل به طور کلی چگونه پردازنده را به پردازه‌های مختلف اختصاص می دهد؟
7) مفهوم File Descriptor در سیستم‌عامل‌های مبتنی بر UNIX چیست؟ عملکرد Pipe در سیستم‌عامل xv6 چگونه است و به طور معمول برای چه هدفی استفاده می شود؟
8) فراخوانی‌های سیستمی exec و fork در سیستم‌عامل xv6 چه عملیاتی را انجام می‌دهند؟ از نظر طراحی، ادغام نکردن این دو چه مزیتی دارد؟

اجرا و برپایی[6] سیستم‌عامل xv6 روی شبیه‌ساز QEMU

در این بخش به اجرای سیستم‌عامل xv6 خواهیم پرداخت. علی‌رغم اینکه این سیستم‌عامل قابلیت اجرا روی سخت‌افزار واقعی را دارد، به منظور فراهم‌کردن محیطی کنترل‌شده و مناسب برای اجرا و اشکال‌زدایی، سیستم‌عامل را به کمک شبیه‌ساز[7] QEMU روی سیستم‌عامل لینوکس اجرا خواهیم کرد. برای این منظور لازم است که کدهای مربوط به سیستم‌عامل xv6 را از لینک ارائه شده clone و یا دانلود کنیم.

یکی از ابزارهای متداول برای کامپایل و ساخت نرم‌افزارهای بزرگ، ابزار make است. نحوه‌ی ساخت پروژه از طریق فایلی به نام Makefile و در قالب قواعد[8]، دستورالعمل‌ها[9] و متغیرها[10] مشخص می‌شود. در xv6 تنها یک Makefile وجود داشته و تمامی فایل‌های سیستم‌عامل نیز در یک پوشه قرار دارند.

در ادامه با اجرای دستور make در پوشه‌ی کد منبع[11] دانلودشده، سیستم‌عامل کامپایل می‌شود و در نهایت با اجرای دستور make qemu، سیستم‌عامل روی شبیه‌ساز اجرا خواهد شد. (توجه شود که فرض شده QEMU از قبل بر روی سیستم‌عامل شما نصب بوده است. در غیر این صورت ابتدا آن را نصب نمایید.)

[2] Single-tasking system
[3] Multitasking system
[4] Program
[5] Process
[6] Setup
[7] Emulator
[8] Rules
[9] Recipes
[10] Variables
[11] Source code

9) ابتدا دستور make clean و سپس دستور make -n را اجرا نمایید. در خروجی، دستوری که فایل نهایی هسته[12] را ایجاد می‌کند پیدا کرده و در گزارش خود ذکر کنید. این دستور از چه ابزاری برای ایجاد فایل نهایی استفاده می‌کند؟

اضافه کردن یک متن به پیام بوت[13]

کد xv6 را به نحوی تغییر دهید که هنگام راه‌اندازی سیستم‌عامل و پیش از نمایش خط فرمان پوسته[14]، نام اعضای گروه به عنوان بخشی از پیام‌های بوت در کنسول نمایش داده شود. تصویر نتیجه را در گزارش کار خود قرار دهید. همچنین در گزارش ذکر کنید برای پیاده‌سازی این قابلیت کدام فایل و کدام بخش از کد را تغییر داده‌اید.

اضافه کردن قابلیت‌های جدید به کنسول و پوسته

در این بخش، چند قابلیت جدید و پرکاربرد را به کنسول و پوسته‌ی xv6 اضافه خواهید کرد:

1. **قابلیت جابه‌جایی مکان‌نما**[15]**:** به کمک این قابلیت، کاربر بتواند مکان‌نما را در رشته‌ی ورودی فعلی کنسول جابه‌جا کرده و متن را بر اساس موقعیت آن ویرایش کند:
   - با فشردن کلیدهای → و ←، مکان‌نما یک کاراکتر[16] به راست یا چپ منتقل شود.
   - با فشردن Ctrl+Q، مکان‌نما به ابتدای رشته‌ی ورودی فعلی منتقل شود.
   - با فشردن Ctrl+E، مکان‌نما به انتهای رشته‌ی ورودی فعلی منتقل شود.
   - عملکرد Backspace باید با موقعیت فعلی مکان‌نما سازگار باشد؛ به این صورت که با فشردن آن، کاراکتر قبل از مکان‌نما حذف شده و کاراکترهای بعد از آن و موقعیت مکان‌نما به‌درستی به‌روزرسانی شوند.

   اگر مکان‌نما در ابتدای رشته قرار داشته باشد، فشردن کلید ← یا Backspace نباید تغییری ایجاد کند. همچنین اگر مکان‌نما در انتهای رشته قرار داشته باشد، فشردن کلید → نباید تغییری ایجاد کند. پس از هر جابه‌جایی یا حذف، متن نمایش‌داده‌شده روی کنسول، موقعیت مکان‌نما و محتوای واقعی بافر ورودی نیز باید با یکدیگر سازگار باشند.

[12] Kernel
[13] Boot message
[14] Shell
[15] Cursor
[16] Character

2. **قابلیت انتخاب[17]، رونوشت[18] و جای‌گذاری[19]:** کاربر بتواند بخشی از رشته‌ی ورودی فعلی کنسول را انتخاب کرده و روی آن عملیات مختلف انجام دهد. نحوه‌ی انتخاب به صورت زیر باشد:
   - با فشردن Ctrl+I در موقعیت فعلی مکان‌نما، نقطه‌ی شروع انتخاب مشخص شود.
   - پس از آن، کاربر تنها با استفاده از کلیدهای مربوط به جابه‌جایی مکان‌نما بتواند موقعیت خود را تغییر دهد.
   - با فشردن مجدد Ctrl+I، نقطه‌ی پایان انتخاب مشخص شده و تمام کاراکترهای بین نقطه‌ی شروع و پایان انتخاب شوند.
   - رشته‌ی انتخاب‌شده باید با رنگ متفاوت یا به صورت Highlight روی کنسول مشخص باشد.

   دقت کنید که ترتیب انتخاب مهم نیست؛ نقطه‌ی دوم می‌تواند قبل یا بعد از نقطه‌ی اول قرار گیرد و در هر دو حالت بازه‌ی بین دو موقعیت باید انتخاب شود. همچنین در صورتی که نقطه‌ی شروع و پایان یکسان باشند، هیچ بخشی از متن انتخاب نشود. پس از انجام انتخاب، عملیات‌های زیر قابل انجام باشند:

   - با فشردن Backspace، کل رشته‌ی انتخاب‌شده حذف شود.
   - با فشردن Ctrl+C، رشته‌ی انتخاب‌شده در حافظه موقت کپی شود و انتخاب همچنان باقی بماند.
   - با فشردن Ctrl+V، رشته‌ی انتخاب‌شده با محتوای کپی‌شده جایگزین شود. در صورتی که هیچ رشته‌ای انتخاب نشده باشد، محتوای کپی‌شده در موقعیت فعلی مکان‌نما قرار گیرد.
   - با وارد کردن یک کاراکتر جدید، رشته‌ی انتخاب‌شده حذف شده و آن کاراکتر در محل انتخاب قرار گیرد.

   پس از انجام هر یک از عملیات فوق، به جز Ctrl+C، حالت انتخاب پایان یافته و کنسول به حالت عادی بازگردد. پس از هر عملیات، متن نمایش‌داده‌شده روی کنسول باید همان محتوای بافر ورودی باشد و موقعیت مکان‌نما نیز با محل واقعی آن در رشته‌ی ورودی مطابقت داشته باشد.

3. **قابلیت نام مستعار[20]:** در بسیاری از Shellها، قابلیت Alias به کاربر اجازه می‌دهد برای یک دستور یا مجموعه‌ای از آرگومان‌های آن، نام کوتاه‌تری تعریف کند. این قابلیت را به Shell سیستم‌عامل xv6 اضافه کنید:

[17] Select
[18] Copy
[19] Paste
[20] Alias

- کاربر باید بتواند با استفاده از قالب alias <name>=<command> یک Alias جدید تعریف کند.
- تمام عبارت موجود پس از علامت = به عنوان دستور متناظر با Alias در نظر گرفته شود.
- پس از تعریف یک Alias، اگر کلمه‌ی اول دستور واردشده دقیقا با نام آن Alias یکسان باشد، فقط همان کلمه با مقدار تعریف‌شده برای Alias جایگزین شود و بقیه‌ی آرگومان‌های دستور بدون تغییر در ادامه‌ی آن باقی بمانند. سپس دستور حاصل به صورت عادی توسط پوسته پردازش و اجرا شود.
- آرگومان‌هایی که کاربر پس از نام Alias وارد می‌کند باید پس از دستور جایگزین‌شده حفظ شوند.
- در صورت تعریف مجدد یک Alias با نامی که از قبل وجود دارد، مقدار قبلی آن با مقدار جدید جایگزین شود.
- Aliasهای تعریف‌شده تنها تا زمان اجرای همان Shell معتبر هستند و نیازی به ذخیره‌ی آن‌ها در فایل یا حفظ آن‌ها پس از راه‌اندازی مجدد xv6 نیست.
- حداقل 10 Alias باید بتوانند به صورت همزمان نگهداری شوند.

برای مثال، پس از تعریف دستور alias search=grep xv6، دستور search README باید معادل اجرای دستور grep xv6 README عمل کند.

4. **قابلیت گسترش تاریخچه[21]:** Shell را به نحوی توسعه دهید که تعدادی از دستورات اخیر واردشده توسط کاربر را نگهداری کرده و امکان اجرای مجدد آن‌ها با استفاده از History Expansion را فراهم کند:
   - Shell باید حداقل 10 دستور اخیر واردشده را به همراه شماره‌ی آن‌ها نگهداری کند.
   - با وارد کردن دستور history، دستورات موجود در تاریخچه به همراه شماره‌ی آن‌ها و به ترتیب زمانی در کنسول نمایش داده شوند.
   - شماره‌ی هر دستور در تاریخچه باید به صورت افزایشی تعیین شود و با حذف دستورات قدیمی از تاریخچه، شماره‌ی سایر دستورات تغییر نکند.
   - با وارد کردن دستور !!، آخرین دستور موجود در تاریخچه مجددا اجرا شود.
   - با وارد کردن دستور !n، که در آن n شماره‌ی یک دستور موجود در تاریخچه است، همان دستور مجددا اجرا شود.
   - با وارد کردن دستور !prefix، آخرین دستوری که ابتدای آن با prefix مشخص‌شده مطابقت دارد، مجددا اجرا شود.

[21] History expansion

- پس از انجام History Expansion، دستور نهایی حاصل باید پیش از اجرا روی کنسول نمایش داده شود.
- دستور حاصل از History Expansion نیز مانند یک دستور عادی اجرا شود و به تاریخچه اضافه گردد.
- در صورت استفاده از History Expansion، دستوراتی مانند !!، n! یا prefix! به عنوان یک دستور مستقل در تاریخچه ذخیره نشوند و تنها دستور نهایی حاصل از Expansion به تاریخچه اضافه گردد.
- در صورتی که دستور متناظری برای History Expansion وجود نداشته باشد، Shell باید پیام خطای مناسبی نمایش داده و هیچ دستوری اجرا نکند.

برای مثال، فرض کنید تاریخچه به‌صورت زیر باشد:

1. echo hello
2. cat README
3. wc README

در این صورت، دستور !! باید به دستور wc README گسترش یافته و اجرا شود. همچنین دستور 2! باید دستور cat README را اجرا کند. دستور ec! نیز باید آخرین دستوری را که با ec شروع می‌شود، یعنی دستور echo hello، اجرا کند.

توجه کنید که در صورت استفاده‌ی همزمان از قابلیت‌های Alias و History Expansion، ابتدا History Expansion روی رشته‌ی ورودی انجام شود و سپس Alias Expansion روی دستور حاصل اعمال گردد. در نهایت دستور نهایی به صورت عادی توسط Shell پردازش و اجرا شود. در هر دستور تنها یک مرحله Alias Expansion انجام شود و نیازی به اعمال بازگشتی Aliasها نیست.

نکته مهم: تمامی قابلیت‌های فوق باید در کنار یکدیگر و به‌طور ترکیبی نیز عملکرد صحیحی داشته باشند. برای مثال، باید بتوان پس از جابه‌جایی مکان‌نما کاراکتری را حذف یا اضافه کرد، رشته‌ای را انتخاب و جایگزین نمود و همچنین از قابلیت‌های Alias و History Expansion در کنار سایر قابلیت‌ها استفاده کرد. هنگام تحویل، چند Test Case ترکیبی نیز بررسی خواهند شد. علاوه بر آن، لازم است Test Caseهایی را که برای آزمون پیاده‌سازی خود استفاده کرده‌اید، به همراه دلیل انتخاب هر کدام در گزارش کار قرار دهید.

برنامه سطح کاربر

در این بخش، هدف آشنایی عملی با نحوه‌ی توسعه‌ی یک برنامه‌ی سطح کاربر در xv6 است. شما در این تمرین با شیوه‌ی دریافت آرگومان‌های خط فرمان، کار با فایل‌ها و استفاده از برخی از فراخوانی‌های سیستمی آشنا می‌شوید. همچنین با اضافه‌کردن یک برنامه‌ی جدید به Makefile، نحوه‌ی قرار گرفتن یک برنامه‌ی سطح کاربر در فرایند ساخت xv6 و اجرای آن در محیط سیستم‌عامل را تمرین خواهید کرد. در ادامه، به پیاده‌سازی یک برنامه‌ی سطح کاربر به زبان C به نام rle می‌پردازید و آن را به برنامه‌های سطح کاربر سیستم‌عامل اضافه می‌کنید.

این برنامه باید عملیات فشرده‌سازی[22] و بازیابی[23] متن را با استفاده از روش Run-Length Encoding انجام دهد. در این روش، کاراکترهای متوالی یکسان به صورت یک زوج شامل خود کاراکتر و تعداد تکرار نمایش داده می‌شوند.

برنامه باید دارای دو حالت زیر باشد:

- در حالت encode، برنامه محتوای فایل ورودی مشخص‌شده در آرگومان برنامه را خوانده و دنباله‌های متوالی از کاراکترهای یکسان را به صورت کاراکتر مربوطه و تعداد تکرار آن در فایل خروجی encoded.txt ذخیره کند.
- در حالت decode، برنامه محتوای فایل ورودی مشخص‌شده در آرگومان برنامه را خوانده و متن اصلی‌اش را بازسازی کرده و در فایل خروجی decoded.txt قرار دهد.

برای جلوگیری از ابهام، هر دنباله‌ی متوالی از یک کاراکتر در قالب خود کاراکتر، سپس یک جداکننده، و سپس تعداد تکرار آن ذخیره شود. برای مثال، رشته‌ی aaabbc به صورت a,3;b,2;c,1; ذخیره شود.

دقت کنید که اجرای decode روی خروجی حاصل از encode دقیقا باید متن اصلی را بازسازی کند. همچنین برنامه باید برای فایل خالی نیز به درستی عمل کند و در صورتی که حالت واردشده معتبر نباشد، تعداد آرگومان‌ها صحیح نباشد یا فایل ورودی قابل باز شدن نباشد، برنامه باید یک پیام خطای مناسب نمایش داده و خاتمه یابد.

شما باید از فراخوانی‌های read، write، open و close در داخل برنامه به درستی استفاده کنید. دقت کنید که برای پیاده‌سازی این برنامه‌ی سطح کاربر، باید در فایل Makefile نیز تغییرات لازم را به وجود آورید تا این برنامه نیز مانند دیگر برنامه‌های سطح کاربر xv6، از جمله ls، قابل اجرا باشد.

[22] Encode
[23] Decode

ورودی و خروجی نمونه در حالت encode:

```
$ cat input.txt
aaabbccccdd
$ rle encode input.txt
$ cat encoded.txt
a,3;b,2;c,4;d,2;
$
```

ورودی و خروجی نمونه در حالت decode:

```
$ cat encoded.txt
a,3;b,2;c,4;d,2;
$ rle decode encoded.txt
$ cat decoded.txt
aaabbccccdd
$
```

10) در فایل Makefile متغیرهایی به نام‌های UPROGS و ULIB تعریف شده‌اند. کاربرد هر یک چیست و چه نقشی در ساخت برنامه‌های سطح کاربر دارند؟

11) اگر به فایل‌های موجود در xv6 دقت کنید، می‌بینید که برخلاف دستوراتی مانند ls و cat، فایل اجرایی مستقلی برای دستور cd وجود ندارد. توضیح دهید که دستور cd در کجا و چگونه پیاده‌سازی شده است. چرا این دستور نمی‌تواند مانند ls یا cat در یک پردازه‌ی فرزند مستقل اجرا شود؟

مراحل بوت سیستم عامل xv6

اجرای بوت‌لودر[24]

هدف از بوت، آماده‌سازی سیستم‌عامل برای سرویس‌دهی به برنامه‌های کاربر است. پس از بوت، سیستم‌عامل سازوکاری جهت ارائه سرویس به برنامه‌های کاربردی خواهد داشت که این برنامه‌ها بدون هیچ مزاحمتی بتوانند از آن استفاده نمایند. کوچکترین واحد دسترسی دیسک‌ها در رایانه‌های شخصی سکتور[25] است. در این‌جا هر سکتور ۵۱۲ بایت است. در فرایند بوت مبتنی بر BIOS، اگر دیسک قابل

[24] Bootloader

[25] Sector

بوت باشد، نخستین سکتور آن، سکتور بوت نام داشته و شامل کد اولیه‌ی Bootloader خواهد بود. Bootloader کدی است که وظیفه‌ی آماده‌سازی اولیه‌ی سیستم و بارگذاری سیستم‌عامل در حافظه را بر عهده دارد. BIOS در صورت یافتن دیسک قابل بوت، سکتور نخست آن را در آدرس فیزیکی 0x7C00 در حافظه قرار داده و اجرای کد موجود در آن را آغاز می‌کند.
توجه کنید که فرایند فوق مربوط به راه‌اندازی مبتنی بر BIOS است. در سیستم‌های مبتنی بر Firmware UEFI به جای اجرای مستقیم نخستین سکتور دیسک، معمولا یک فایل اجرایی با قالب EFI را از EFI System Partition پیدا کرده، آن را در حافظه بارگذاری کرده و اجرا می‌کند. بنابراین مفاهیمی مانند Boot Sector و بارگذاری کد در آدرس 0x7C00 مربوط به روش Legacy BIOS هستند و در UEFI روند بوت متفاوت است.

12) در xv6 چه فایل ساخته‌شده‌ای در نخستین سکتور دیسک قابل بوت قرار می‌گیرد؟ این فایل از کدام فایل‌های کد منبع ساخته می‌شود؟ (راهنمایی: خروجی دستور make -n و فایل Makefile را بررسی نمایید.)
13) فایل bootblock نهایی چه نوع فایل دودویی[26] است؟ این فایل چه تفاوتی با فایل‌های ELF مورد استفاده در سایر بخش‌های xv6 دارد و چرا برای بوت سکتور از این قالب استفاده شده است؟ همچنین محتوای اسمبلی[27] bootblock.o را با استفاده از objdump بررسی کرده و بخشی از آن را با فایل bootasm.S مقایسه نمایید.
14) علت استفاده از دستور objcopy در حین اجرای عملیات make چیست؟
15) فرایند بوت xv6 توسط فایل‌های bootasm.S و bootmain.c انجام می‌شود. چرا نمی‌توان اجرای Bootloader را مستقیما با کد C آغاز کرد؟ نقش کلی هر یک از این دو فایل را توضیح دهید.

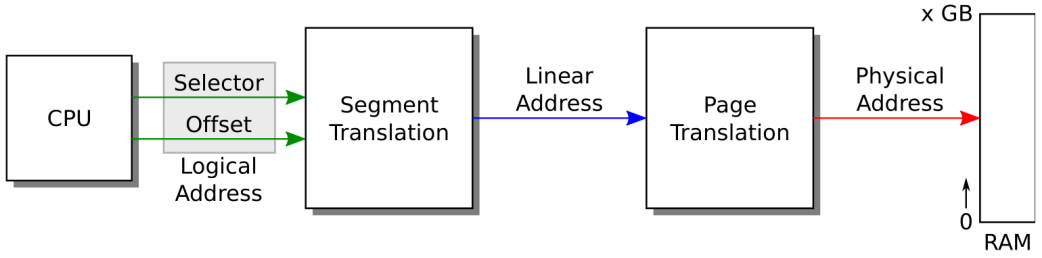
معماری سیستم شبیه‌سازی شده x86 است. حالت سیستم در حال اجرا در هر لحظه را به طور ساده می‌توان شامل حالت پردازنده و حافظه دانست. بخشی از حالت پردازنده در ثبات‌های آن نگه‌داری می‌شود.

وضعیت ثبات‌ها را می‌توان به کمک gdb و دستور info registers مشاهده نمود. وضعیت برخی از ثبات‌های دیگر نیاز به دسترسی ممتاز[28] دارد. به این منظور می‌توان از QEMU استفاده نمود. کافی است با زدن Ctrl + A و سپس C به ترمینال QEMU رفته و دستور info registers را وارد نمود. با تکرار همان دکمه‌ها می‌توان به xv6 بازگشت.

[26] Binary
[27] Assembly
[28] Privileged Access

16) یکی دیگر در از مدهای مهم، مد حفاظت‌شده[29] می‌باشد. وظیفه‌ی اصلی این مود چیست؟ پردازنده‌ها در چه زمانی در این مود قرار می‌گیرند؟

در ابتدا QEMU یک هسته را جهت اجرای کد بوت bootasm.S فعال می‌کند. فرایند بوت در بالاترین سطح دسترسی[30] صورت می‌گیرد. به عبارت دیگر، بوت‌لودر امکان دسترسی به تمامی قابلیت‌های سیستم را دارد. در ادامه هسته به مد حفاظت‌شده تغییر مد می‌دهد (خط ۹۱۵۳). در مد حفاظت‌شده، آدرس مورد دسترسی در برنامه (آدرس منطقی) از طریق جداولی به آدرس فیزیکی حافظه[31] نگاشت پیدا می‌کند.
ساختار آدرس‌دهی در این مد در شکل زیر نشان داده شده است:



17) چرا برای بارگذاری اصلی هسته از محدوده‌ی فیزیکی 0x100000 استفاده شده است[32] و هسته از آدرس فیزیکی صفر بارگذاری نمی‌شود؟ همچنین تفاوت نقش آدرس‌های 0x10000 و 0x100000 را در فرایند boot توضیح دهید.

در ادامه و در انتهای بوت، کد هسته سیستم‌عامل به طور کامل در حافظه قرار خواهد گرفت. در گام انتهایی نیز، بوت‌لودر اجرا را به هسته واگذار می‌نماید. باید کد ورود به هسته اجرا گردد. این کد اسمبلی در فایل entry.S قرار داشته و نماد (بیانگر مکانی از کد) entry از آن فراخوانی می‌گردد. آدرس این نماد در هسته بوده و حدود 0x100000 است.

[29] Protected Mode

[30] سطوح دسترسی در ادامه پروژه توضیح داده خواهد شد.

[31] منظور از آدرس فیزیکی یک آدرس یکتا در سخت‌افزار حافظه است که پردازنده به آن دسترسی پیدا می‌کند.

[32] دقت شود آدرس 0x100000 تنها برای خواندن هدر فایل elf استفاده شده است و محتوای فایل هسته در 0x100000 که توسط paddr (مخفف آدرس فیزیکی) تعیین شده است، کپی می‌شود. این آدرس در زمان لینک توسط kernel.ld تعیین شده و در فایل دودویی در قالب خاصی قرار داده شده است.

اشکال‌زدایی

کد هر برنامه‌ای ممکن است دارای اشکال باشد. اشکال‌زدایی ممکن است ایستا[33]، پویا[34] و یا به صورت ترکیبی[35] صورت پذیرد. کشف اشکال در روش‌های ایستا، بدون اجرا و تنها بر اساس اطلاعات کد برنامه صورت می‌گیرد. به عنوان مثال کامپایلر Clang دارای تحلیل‌گرهای ایستا[36] برای اشکال‌زدایی اشکال‌های خاص است. اشکال‌زدایی پویا که معمولا دقیق‌تر است، اقدام به کشف اشکال در حین اجرای برنامه می‌نماید. ابزار leak-check در ابزار Valgrind یک اشکال‌زدای پویا برای تشخیص نشتی حافظه[37] است.

از یک منظر می‌توان اشکال‌زداهای پویا را به دو دسته تقسیم نمود:

1. اشکال‌زداهایی که برای شناسایی نوع خاصی از خطاها، مانند نشتی حافظه[38]، طراحی شده‌اند.
2. اشکال‌زداهایی که مستقل از نوع اشکال بوده و تنها اجرا را ردگیری[39] نموده و اطلاعاتی از حالت سیستم (شامل سخت‌افزار و نرم‌افزار) در حین اجرا یا پس از اجرا جهت درک بهتر رفتار برنامه برمی‌گردانند.

در این بخش ابزار اشکال‌زدای گنو[40] (GDB)، که یک اشکال‌زدای پویا از نوع دوم است معرفی خواهد شد.

GDB یک اشکال‌زدای متداول در سیستم‌های یونیکسی بوده که در بسیاری از شرایط، نقش قابل‌توجهی در تسریع روند اشکال‌زدایی ایفا می‌کند. اشکال‌زدایی برنامه‌های تک‌ریسه‌ای[41]، چندریسه‌ای[42] و حتی هسته‌های سیستم‌عامل توسط این ابزار ممکن است.

جهت اشکال‌زدایی xv6 با GDB، ابتدا باید QEMU را به گونه‌ای اجرا کرد که در ابتدای اجرا متوقف شده و منتظر اتصال GDB بماند. مراحل کلی اتصال به صورت زیر است:

1. در یک ترمینال دستور make qemu-gdb را اجرا کنید. این دستور QEMU را با قابلیت اتصال GDB اجرا کرده و فایل gdbinit. مورد نیاز را نیز ایجاد می‌کند.
2. در ترمینال دیگری و در پوشه‌ی xv6، حالا GDB را اجرا کنید. فایل gdbinit. ایجادشده شامل اطلاعات لازم برای اتصال به QEMU و بارگذاری Symbolهای فایل kernel است.
3. در صورتی که GDB به دلایل امنیتی از بارگذاری خودکار فایل gdbinit. جلوگیری کرد، پیام نمایش‌داده‌شده توسط GDB را بررسی کرده و تنظیمات لازم را مطابق آن انجام دهید. همچنین

[33] Static
[34] Dynamic
[35] Hybrid
[36] Static analyzer
[37] Memory Leak
[38] Memory leakage
[39] Tracing
[40] GNU Debugger
[41] Single-Thread
[42] Multithread

می‌توانید شماره‌ی Port مورد استفاده برای اتصال GDB را در فایل gdbinit. یا خروجی Makefile مشاهده کنید.

روند اجرای GDB

GDB می‌تواند در هر گام از اجرا، با ارائه حالت سیستم، به برنامه‌نویس کمک کند تا حالت خطا را از حالت مورد انتظار تشخیص دهد. هنگام اجرای کد در GDB ممکن است چندین حالت رخ دهد:

1. اجرا با موفقیت جریان داشته باشد یا خاتمه یابد.
2. اجرا به علت اشکال، ناتمام مانده و برنامه متوقف شود.
3. اجرا متوقف نشده ولی حالت سیستم در برخی نقاط درونی یا در خروجی‌های برنامه نادرست باشد.

هدف، یافتن حالات خطای سیستم در دو وضعیت ۲ و ۳ است. به عبارتی ابتدا باید در نقطه مورد نظر، توقف صورت گرفته و سپس به کمک دستورهایی حالت سیستم را استخراج نمود. برای توقف اجرا در نقاط مختلف اجرا در GDB سازوکارهای مختلفی وجود دارد:

1. در اجرای ناتمام، اجرای برنامه به طور خودکار متوقف می‌شود.
2. با فشردن کلید ترکیبی Ctrl + C به اشکال‌زدا بازگشت.

این عملیات در میان اجرا، آن را متوقف نموده و کنترل را به خط فرمان اشکال‌زدا منتقل می‌کند. مثلاً حلقه بی‌نهایت رخ داده باشد، می‌توان با این کلید ترکیبی، در نقطه‌ای از حلقه متوقف شد.

3. روی نقطه‌ای از برنامه Breakpoint قرار داد. بدین ترتیب هر رسیدن اجرا به این نقطه منجر به توقف اجرا گردد.

روش‌های مختلفی برای تعیین نقطه استقرار Breakpoint وجود داشته که در این لینک قابل مشاهده است. از جمله:

انتخاب نام و شماره خط فایل

```
$ break cat.c:12
```

انتخاب نام تابع

```
$ b cat
```

انتخاب آدرس حافظه

```
$ b *0x98
```

این نقاط می‌توانند در سطح کاربر یا هسته سیستم‌عامل باشند. همچنین می‌توانند شرطی تعریف شوند.

4. روی خانه خاصی از حافظه Watchpoint قرار داد تا دسترسی یا تغییر مقدار آن خانه، منجر به توقف اجرا گردد.

Watchpoint ها انواع مختلفی داشته و با دستورهای خاص خود مشخص می‌گردند.

دستور زیر:

```
$ watch *0x1234567
```

یک Watchpoint روی آدرس 0x1234567 در حافظه می‌گذارد. بدین ترتیب نوشتن در این آدرس، منجر به توقف اجرا خواهد شد.

می‌توان از نام متغیر هم استفاده نمود. مثلاً watch v، Watchرا روی (آدرس) متغیر v قرار می‌دهد.

باید دقت نمود، اگر Watch روی متغیر محلی قرار داده شود، با خروج از حوزه دسترسی به آن متغیر، Watch حذف شده و به برنامه‌نویس اطلاع داده می‌شود. اگر هم آدرسی از فضای پشته[43و44] داده شود، ممکن است در حین اجرا متغیرها یا داده‌های نامرتبط دیگری در آن آدرس نوشته شود. یعنی این آدرس در زمان‌های مختلف مربوط به داده‌های مختلف بوده و در عمل Watch کارایی مورد نظر را نداشته باشد.

یک مزیت مهم Watch، تشخیص وضعیت مسابقه[45] است که در فصول بعدی درس با آن آشنا خواهید شد. در این شرایط می‌توان تشخیص داد که کدام ریسه[46] یا پردازه مقدار نامناسب را در آدرس حافظه نوشته که منجر به خطا شده است.

همان‌طور که مشاهده می‌شود، خیلی از حالات با استفاده از چهار سازوکار مذکور به سهولت قابل استخراج نیستند. مثلاً حالتی که یک زنجیره خاص فراخوانی توابع وجود داشته باشد یا این که مثلاً حالتی خاص در داده‌ساختارها رخ داده و یک لیست پیوندی، چهارمین عنصرش را حذف نماید.

20) برای مشاهده Breakpoint ها از چه دستوری استفاده می‌شود؟

21) برای حذف یک Breakpoint از چه دستوری و چگونه استفاده می‌شود؟

کنترل روند اجرا و دسترسی به حالت سیستم

پس از توقف می‌توان با استفاده از دستورهایی به حالت سیستم دسترسی پیدا نمود. همچنین دستورهایی برای تعیین شیوه ادامه اجرا وجود دارد. در ادامه، برخی از دستورهای کنترلی و دسترسی به حالت اجرا معرفی خواهد شد.

پس از توقف روی Breakpoint، می‌توان با استفاده از دستورهای next، step و finish روند اجرا را به صورت گام‌به‌گام بررسی کرد. دستور step خط بعدی برنامه را اجرا کرده و در صورت وجود فراخوانی تابع، وارد آن تابع می‌شود؛ در حالی که دستور next فراخوانی تابع را به صورت یک گام اجرا کرده و وارد جزئیات داخلی آن نمی‌شود. دستور finish نیز اجرای تابع جاری را تا زمان بازگشت به تابع فراخواننده

[43] Stack

[44] یعنی فضای آدرسی که داده‌هایی از جمله مقادیر متغیرهای محلی و آدرس‌های برگشت مربوط به توابع فراخوانی شده در آن قرار دارد.

[45] Race Condition

[46] Thread

ادامه می‌دهد. دستور continue اجرای برنامه را تا رسیدن به نقطه‌ی توقف بعدی یا پایان اجرا ادامه می‌دهد و با دستور list نیز می‌توان کد منبع اطراف محل توقف را مشاهده کرد.

22) دستور زیر را اجرا کنید. خروجی آن چه چیزی را نشان می‌دهد؟

```
$ bt
```

23) دو تفاوت دستورهای x و print را توضیح دهید. چگونه می‌توان محتوای یک ثبات خاص را چاپ کرد؟ (راهنمایی: می‌توانید از دستور help استفاده نمایید: help x و help print)

24) برای نمایش وضعیت ثبات‌ها و متغیرهای محلی در GDB از چه دستوراتی استفاده می‌شود؟ نتیجه‌ی این دستورها را در گزارش خود بیاورید. همچنین کاربرد متعارف ثبات‌های ESI و EDI در معماری x86 را توضیح دهید.

25) با استفاده از GDB، ساختار input در فایل console.c را بررسی کرده و موارد زیر را در گزارش توضیح دهید:

- نقش کلی این ساختار و متغیرهای r، w و e در نسخه‌ی اولیه‌ی xv6 چیست؟
- هنگام وارد کردن چند کاراکتر و سپس فشردن Enter، مقادیر این متغیرها چگونه تغییر می‌کنند؟
- برای پیاده‌سازی قابلیت‌های جدید کنسول چه State یا متغیرهای جدیدی به پیاده‌سازی خود اضافه کرده‌اید و هر کدام چه نقشی دارند؟
- یک Breakpoint مناسب در مسیر دریافت ورودی قرار داده و مقادیر مربوط به رشته‌ی ورودی و موقعیت مکان‌نما را در یک نمونه‌ی دلخواه با GDB بررسی کنید. نتیجه را در گزارش قرار دهید.

اشکال‌زدایی در سطح کد اسمبلی

اشکال زدایی برنامه در سطوح مختلفی قابل انجام است. با توجه به این که بسیاری از جزئیات اجرا در کد سطح بالا (زبان C) قابل مشاهده نیست، نیاز به اشکال‌زدایی در سطح کد اسمبلی خواهد بود. به عنوان مثال بهینه‌سازی‌های ممکن است ترتیب اجرا در کد سطح بالا را تغییر داده یا بخشی از کد را حذف نماید. به عنوان مثال دیگر می‌توان از شیوه دسترسی به جداول لینکر نام برد. جزئیات دسترسی به یک تابع کتابخانه‌ای خاص یا یک متغیر سراسری آن کتابخانه دسترسی شده است، در سطح کد اسمبلی و با دسترسی به جداول لینک رخ داده و در سطح زبان سی قابل رؤیت نیست.

با فشردن Ctrl+X و سپس A می‌توان رابط کاربری متنی[47] GDB یا همان TUI را فعال یا غیرفعال کرد و کد اسمبلی مربوط به نقطه توقف را مشاهده کرد. برای اطلاعات بیشتر در رابطه با این رابط کاربری می‌توانید به این صفحه مراجعه کنید.

[47] Text user interface

26) خروجی دستورات layout src و layout asm در TUI چیست؟
27) برای جابه‌جایی میان توابع زنجیره فراخوانی جاری (نقطه توقف) از چه دستوراتی استفاده می شود؟

دستورهای stepi و nexti معادل‌های سطح اسمبلی step و next بوده و به جای یک دستور سی، در ریزدانگی یک دستورالعمل ماشین عمل می‌کنند. در شرایطی که کد مورد اشکال‌زدایی از ابتدا در زبان اسمبلی نوشته شده باشد، چاره‌ای جز استفاده از این دستورها وجود نخواهد داشت.

نکات پایانی

با توجه به کاستی‌هایی که در اشکال‌زداها وجود دارد، همچنان برخی از تکنیک‌ها در کدزنی می‌تواند بسیار راهگشا باشد. ساده‌ترین راه برای اشکال‌زدایی این است که تغییرها را اندک انجام داده و گام‌به‌گام از صحت اجرای کد، اطمینان حاصل شود. به عنوان مثال اگر آرایه‌ای ۱۰۰ عنصری تخصیص داده شده و در نقطه‌ای فراتر از مرز انتهایی آن نوشتن صورت گیرد، حافظه‌ای غیر از حافظه مربوط به آرایه دستکاری می‌گردد. چندین حالت ممکن است رخ‌دهد، از جمله اینکه:

1. اقدام به نوشتن در حافظه‌ای فقط خواندنی مانند کد برنامه، صورت پذیرد. در چنین شرایطی خطا رخ داده و نقطه توقف به راحتی در GDB قابل رؤیت خواهد بود.
2. در حافظه نوشتنی نامرتبط نوشته شده و مشکلی پیش نیاید.
3. در حافظه نوشتنی نامرتبط نوشته شود و اجرای برنامه به طرز عجیبی متوقف گردد. به طوری که GDB نقطه نامربوطی را نشان دهد. یعنی تأثیر آن بلافاصله و به طور مستقیم رخ ندهد. در چنین شرایطی استفاده ابتدایی از اشکال‌زدا راحتی راهگشا نخواهد بود. چک کردن اندازه آرایه و احتمال دسترسی به خارج آن در سطح کد، می‌توانست راحت‌تر باشد. البته در برخی موارد به سادگی و یا با تکنیک‌هایی مانند استفاده از Watch، ضبط اجرا و حرکت رو به عقب از حالت نادرست، می‌توان اشکال را یافت[48]. اما تکنیک قبلی بهتر بود.

بنابراین، استفاده از GDB در کنار دیگر ابزارها و تکنیک‌ها در پروژه‌های این درس توصیه می‌گردد. با توجه به آشنایی اولیه‌ای که با GDB فراهم شده است، می‌توان مزایای آن را برشمرد:

- اشکال‌زدایی کدهای بزرگ و کدهایی که با پیاده‌سازی آن‌ها آشنایی وجود ندارد. ممکن است نیاز باشد یک کد بزرگ را به برنامه اضافه کنید. در این شرایط اشکال‌زدایی اجرای Crash کرده در GDB درک اولیه‌ای از نقطه خرابی ارائه می‌دهد.
- بررسی مقادیر حالت برنامه، بدون نیاز به قرار دادن دستورهای چاپ مقادیر در کد و کامپایل مجدد آن.

[48] GDB در برنامه‌های عادی قادر به ضبط و اجرای رو به عقب برنامه است. همچنین ابزار RR که توسط شرکت موزیلا برای اشکال‌زدایی فایرفاکس ارائه شده است امکان انجام همین عملیات را به صورت قطعی دارد. این قطعیت، در اشکال‌زدایی کدهای همروند و وضعیت مسابقه بسیار کمک‌کننده است.

- بررسی مقادیر حالت سخت‌افزار و برنامه که در سطح کد قابل رویت نیستند. به عنوان مثال مقدار یک اشاره‌گر به تابع، مقصد یک تابع کتابخانه‌ای، اطمینان از قرارگیری آدرس متغیر محلی در بازه حافظه پشته، این که اجرا در کدام فایل کد منبع قرار دارد، اطلاع از وضعیت فضای آدرس حین اجرا، مثلا این که هر کتابخانه در چه آدرسی بوده و در کدام کتابخانه در حال اجرا هستیم.

نکات مهم

- پروژه خود را در یک مخزن خصوصی در GitHub پیش برده و در نهایت **یک نفر** از اعضای گروه کدها را به همراه پوشهٔ .git[49] و فایل pdf گزارش کار زیپ کرده و در سامانه با فرمت OS-Lab1-<SID1>-<SID2>-<SID3>.zip آپلود نمایید. **رعایت نکردن این مورد موجب کسر نمره خواهد شد.**
- به تمامی سوالات **به طور خلاصه** پاسخ داده و پاسخ‌ها و نتایج را در گزارش کار خود قید کنید.
- بخش خوبی از نمره‌ی شما را پاسخ‌دهی به سوالات مطرح شده تشکیل می‌دهد که به شما در درک نحوه‌ی کارکرد xv6 کمک می‌کند.
- مطالعه‌ی فصل‌های صفر و یک و پیوست ب از کتاب xv6 **الزامی** بوده و مطالعه‌ی پیوست الف نیز توصیه می‌شود.
- آشنایی کامل و عملی با GDB یکی از اهداف اصلی این پروژه است. در پروژه‌های بعدی درس، از این ابزار به‌طور گسترده برای بررسی و اشکال‌زدایی پیاده‌سازی‌ها استفاده خواهد شد. بنابراین توصیه می‌شود علاوه بر پاسخ به سوالات این بخش، نحوه‌ی استفاده از قابلیت‌های اصلی GDB را به‌خوبی تمرین کنید.
- در پیاده‌سازی خود در خصوص مواردی که ذکر نشده‌اند می‌توانید فرضی منطقی و متناسب با رفتار پوسته‌ی لینوکس در نظر بگیرید.
- در صورت وجود هرگونه سوال یا ابهام می‌توانید با ایمیل یا تلگرام دستیاران مربوطه و یا گروه درس در ارتباط باشید.
- همه‌ی اعضای گروه باید به تمام بخش‌های پروژه‌ی آپلود شده توسط گروه، چه کد و چه سوالات، مسلط بوده و **هنگام تحویل از تمامی اعضا و از تمامی قسمت‌ها به صورت تصادفی سوال پرسیده خواهد شد.**

[49] برای اطمینان حاصل کردن از وجود پوشه git. می‌توانید گزینه‌ی نمایش فایل‌های مخفی (Show Hidden Files) را فعال و یا از دستور ls -a استفاده کنید.

- در صورت مشاهده‌ی هرگونه شباهت بین کدها یا گزارش‌ها میان گروه‌ها در این ترم یا ترم‌های گذشته، مطابق با سیاست‌های درس برخورد خواهد شد.

موفق باشید.